Final Project: Pick a Project

Computer Science - Week 10

Oct 3, 2026 - Version 0.0.2

Please make sure to your UD **email** AND **name** is below. Unless your UD emails are included in this table, then you will not earn any points for this assignment when it is graded! **Every person should submit their own individual worksheet**.

Talk about each of the questions with a friend or our LLM (or both), but **make sure the words you enter below are yours** (with the obvious exceptions notwithstanding). Brainstorming is best done in full sentences, and the main goal here is brainstorming. It's worth reviewing the information in this series of pages in the Drafter documentation to gain a better understanding of the process.

Role	Name	Email
Developer (You)	Nathan McGaffic	nmcgaffi@udel.edu
Friend (optional)	Seth Blomquist	sethblom@udel.edu

Part 1: The Goal & Prior Implementations

1. **Identify the Goal:** What specific goal are you trying to address? Think about your passions, hobbies, and academic interests. The latter are an excellent candidate to focus on. Some majors have suggestions listed below: these were provided by faculty in those departments. Other majors could consider a focus tied to a big final assignment in one of their recent courses in their major.
    - **Biomedical Engineering** majors may consider one of the following:
        - If you have started (or completed) the Cell/Tissue Design final project in BMEG100, design an organoid, which involves combining a cell type, biomaterial, and testing method. Have the website include a description and diagram of your design concept, a testing method, a needs statement, and an impact statement relevant to their organoid. Also include a link to a paper that supports your idea and a paragraph describing how that paper supports your design concept.

        - Otherwise, create a website related to your BMEG100 Lab Report 1, *Cytoskeletal and Nuclear Staining Lab Report Guidelines*, linked here (University of Delaware account required to access).
    - **Chemistry** and **Chemical Engineering** majors may consider creating an automated chemistry assistant that takes balanced reaction equations, starting reactant masses, and a harvested product amount to determine the limiting reactant, theoretical yield, and percent yield based on principles from Chemistry 2e (OpenStax) — Chapter 4: Chemical Reactions and Stoichiometry. To avoid manually compiling atomic mass tables or writing custom string tokenizers, use the ChemPy library to parse molecular formulas into exact molecular weights and verify reaction stoichiometry against compound data from the NIH PubChem Database. The program outputs an organized breakdown of limiting reagent status, product yields, and remaining masses of unreacted species. Also use the heat of the reaction to determine if it is exothermic or endothermic, and compute the heat flow per mole of the product.
    - Majors in the **Electric & Computer Engineering** department may consider a steganography project that hides messages in images, and then retrieves those messages from uploaded images.
    - **Mechanical Engineering** majors may consider a project that somehow uses both the NumPy and SciPy Python libraries.

I want to make a video game recommendation web application that takes many of the users preferences like game genre, maximum price, platform (pc or a specific console), and the minimum review rating the game can have. The game should then filter through all the games that are out and recommend the game with the best value to the user. Also if there is no game in the first genre that has all of the original requirements, the program will already have asked for a second choice genre.

2. **Prior Implementations:** Search the internet. What existing tools or websites currently attempt to address it?

Steam filters, quantic foundry, and built in stores for consoles already exist to try and help people find good game recommendations.

3. **Your Implementation:** Briefly describe the legitimate purpose or functionality of your site. How would you want this web application to be used?

An application that is hopefully easier to find good games. This implementation will allow

video game players to input their specific preferences so they can quickly get a recommendation made specially for them that is within their price range.

Part 2: The Input-Process-Output (IPO) Model

Think about the fundamental flow of your program.

1. **The Core Goal:** Every program has three main phases: taking input, processing data, and presenting output. In one sentence, what is the ultimate goal your program will accomplish for the user? Be specific and use CS1 concepts.

The program takes user game preferences as the input, processes data from a set of data and filters it based on conditional statements to find something that the user would like, and then returns the top game suggestion or the suggestion of a game that falls within their second choice genre.

2. Input: What specific information will the user need to provide to your program? (e.g., a city name, a numerical budget, a list of hobbies).

The user will input the preferred genre, maximum price, the platform (pc or specific console), the minimum rating a game should have, and the option for another game genre in case their first genre does not have a game that meets their requirements.

3. Processing: At a high level, how will the computer manipulate or evaluate that data? (e.g., calculate the average, filter out options that are too expensive, compare the input against a database).

The program will go through a list of games and filter out games that do not match the selected platform, genre, maximum price, and minimum rating. It will then look through all of the games that are left and find one with the lowest price and highest rating. If there are no games left after the search for the preferences a secondary search using the backup genre will be done.

4. Output: What will the user get back? (e.g., a recommendation, a calculated safety factor, a formatted schedule).

The recommended game's title, platform, price, rating, and genre details. If it cannot find one for the first choice a message will say that no game for the original search could be

found.

Part 3: Data Abstraction

Identify the "nouns" of your problem.

1. **Representing Reality:** Abstraction is the process of representing the real world inside a computer by ignoring unnecessary details. What are the core "things" (nouns) your program needs to track? (e.g., *If building a recipe app, the nouns are "Recipe" and "Ingredient"*).

Game

2. **Identifying Properties:** For the most important "thing" you listed above, what 3 to 4 specific details do you actually need to solve your problem, and what details will you abstract away?
    - *Details I need:* (e.g., *Ingredient name, weight, cost*)
    - *Details I will ignore:* (e.g., *Ingredient color, brand, expiration date*)

Game title, platform, price, and rating

Part 4: Challenges, AI Strategy, & Goals

1. **Difficulties:** What parts of this project seem the most difficult or technically complex?

Setting up the condition that switches to another genre if no game from the first genre could be found.

2. **Coding Strategy:** If you will be using an LLM to help code, how will you ensure you understand the generated code and avoid "hallucinations" or "shallow learning" (copying without understanding)? If you won't be using an LLM to help code, this project might take about 4 hours a week to complete, so list each of the days and times in which you work on the project.

I will use an LLM to help me understand the errors that come up and to make sure I am

following all syntax rules. I will write out the main functions and the unit tests to make sure I understand the code.

3. **Goals:** What is your ultimate goal for this project? (e.g., *Impress the graders with a creative UI, build something I can use in my other classes, learn Drafter deeply*).

To make a working tool that I could actually use to find games that have all of the things that I would prefer.

Part 5: Research & Brainstorming Conversation

Have a conversation with a Generative AI tool (like ChatGPT or Claude) OR a friend about your project idea. Do not ask for code. Instead, use the AI/friend as a sounding board to research your problem.

1. **The Initial Prompt/Question:** Ask the AI or your friend to help you brainstorm. (e.g., *"I want to build a simple web tool to help students track their study hours. What are 3 common problems students face when tracking their time, and what are 3 simple features a basic tool should have?"*). Paste your prompt below:

I want to build a video game recommendation web tool. What are 3 common problems people come across when browsing digital game stores, and what are 3 of the main features a basic recommendation tool should have?

2. **The Response:** Briefly summarize the best ideas the AI or your friend suggested. What existing solutions or prior apps did they mention?

The AI suggested that common problems are “analysis paralysis” from too many choices, prices exceeding personal budgets, and platform incompatibility. Key features recommended were budget filtering, platform tagging, and rating sorting.

3. **The Pushback:** You must reply to the AI/friend to narrow the scope when it's over-ambitious. Pick the most complex feature they suggested and explain why it is too complicated for an introductory programming project. Ask them for a simpler alternative. *Paste your response here:*

Trying to connect to other game stores like Steam and PlayStation to get current prices in

real time is too complex for me to be able to do at this level because there are a lot of video games in the world.

4. **Reflection:** Based on this conversation, how has your project idea changed or become more focused?

This conversation helped me understand that I should use a small dataset instead of trying to make something that looks all over the internet for games. It helped me realize I should just use a smaller dataset like in the lead_in_water project so that I can still have a lot of games but not have to find out how to connect my project to the web.

Part 6: Resource Planning

1. What resources will you use if you get stuck?

Course chapters, drafter website instructions, help from classmates, and just looking at the errors and where they are coming from.

2. If your LLM provides code that causes a Drafter or Bakery error, or if you run into a problem during deployment, what is your step-by-step debugging plan to resolve it?

I will look at the line number where the error took place and test the specific function to see if the function returns the value it is supposed to.

Part 7: LLM Conversation

If you used an LLM in any part of this work, copy and paste every part of that conversation below. Otherwise, you may be given a zero for the assignment. Obviously, these words will not all be yours, and this is the exception to the requirement at the top of this assignment that all responses be in your own words. If you did not use an LLM to assist you in this assignment, simply say so below.

Me:

I want to build a web tool that will make a video game recommendation web application that takes many of the users preferences like game genre, maximum price, platform (pc or a specific console), and the minimum review rating the game can have. The game should then filter through all the games that are out and recommend the game with the best value to the user. Also if there is no game in the first genre that has all of the original requirements, the program will already have asked for a second choice genre.

AI:

Building a video game recommendation web application that balances personal preferences with the concept of **"best value"** is a fantastic project! It combines database filtering, preference logic, and a scoring system to determine what "value" actually means (e.g., playtime per dollar, review score relative to price, or discounts).

Here is a comprehensive guide and blueprint on how to design, architect, and build your web tool.

1. Core Features & User Flow

- **Step 1: Preference Input Form** (Platforms, Genres with primary/secondary choices, Budget, Minimum Review Rating)
- **Step 2: The "Best Value" Algorithm** (Filtering master database and calculating value score)
- **Step 3: Fallback Logic** (Automatically running the filter using the second choice genre if the primary yields zero results)

2. Suggested Tech Stack

- **Frontend:** React, Vue.js, or HTML/Tailwind CSS with Vanilla JS.
- **Backend:** Node.js (Express) or Python (FastAPI / Flask).
- **Data Source / API:** RAWG Video Games Database API, IGDB, or Steam Store API.

(I have no idea what this stuff at the bottom is but the AI started talking about it)

Part 8: Submit

In Google docs, with this document open, please click on *File* > *Download* > *Microsoft Word (.docx)*, and make sure you put that download somewhere you can find it again. Then, upload it to the Canvas assignment about selecting or brainstorming your final project.

When you're done with this assignment, review this page of the Drafter documentation to better prepare for the next step.